COMPSCI 308 Design and Analysis of Algorithms

Quiz 01

Xing Shi Cai

2026-02-02

👀 Don't Cheat

During the quiz/exam do *not*

- 🩹 give or receive aid;
- ⏱️ read the questions before the quiz begins;
- 🛑 keep writing when the quiz ends;
- 📱 use a phone, calculator, computer or the internet;
- 🚽 hide textbooks or other aids in the toilet.

⚡ Follow Rules

- 👨‍🎓 Enter your name and **SID** (not your Net ID) in the designated space below.
- 🆎 Support your proofs with both equations and descriptive reasoning.
- ✏️ Use only pen for writing as pencil marks are not clearly visible.
- ❓ Restrict your inquiries to clarifications regarding typos.

Name: SID:

Figure 1: Good Luck

1 Greedy Algorithm (1pt)

The standard greedy algorithm for the Activity Selection Problem selects the activity with the *earliest finish time* compatible with those already selected. Suppose that instead, we select the activity with the **latest start time** that is compatible with all previously selected activities.

1.1 (0.4pt)

Run this algorithm on the following set of activities. List the indices of the selected activities in increasing order.

- Activity 1: $[1, 4)$
- Activity 2: $[3, 5)$
- Activity 3: $[0, 6)$
- Activity 4: $[5, 7)$
- Activity 5: $[3, 8)$
- Activity 6: $[6, 10)$
- Activity 7: $[8, 11)$
- Activity 8: $[2, 14)$

Answer:

1.2 (0.2pt)

Explain in one sentence why this strategy is considered a **greedy** strategy.

💡 Focus on the local choice property of a greedy algorithm.

Answer:

1.3 (0.4pt)

Prove that the algorithm yields an optimal solution.

💡 Hint: Consider the symmetry between this problem and the standard activity selection problem. One sentence is sufficient.

Answer:

2 Huffman Code — Lemma 15.3 (1pt)

Lemma 15.2

For the least frequent characters $x, y \in C$, there exists an optimal prefix code where codewords for x and y have equal length and differ only in the last bit.

Lemma 15.3

Let $x, y \in C$ be the characters with minimum frequencies.
Let $C' = C - \{x, y\} \cup \{z\}$ where $z.freq = x.freq + y.freq$.
For a tree T' representing an optimal prefix code for C', the tree T formed by replacing T''s leaf node for z with an internal node having x and y as children, yields an optimal prefix code for C.

We will prove Lemma 15.3 by following the steps below.

2.1 (0.2pt)

Cost Relationship: Show that $B(T) = B(T') + x.freq + y.freq$.

Answer:

2.2 (0.2pt)

Contradiction Assumption: To prove by contradiction, what assumption should we make about T? What does this imply about the existence of another tree T''?

Answer:

2.3 (0.2pt)

Applying Lemma 15.2: How can we use Lemma 15.2 to refine our choice of the optimal tree T''?

Answer:

2.4 (0.2pt)

Constructing a Better Tree for C': How can we construct a tree T''' for C' from T''?

Answer:

2.5 (0.2pt)

Contradiction: Show that $B(T''') < B(T')$, and explain why this is a contradiction.

Answer:

3 Transpose of a Graph (1pt)

The **transpose** of a directed graph $G = (V, E)$ is the graph $G^T = (V, E^T)$, where $E^T = \{(v, u) \in V \times V : (u, v) \in E\}$. That is, G^T is G with all its edges reversed.

We want to compute G^T from G.

3.1 (0.4pt)

Adjacency List (Pseudocode): Given the adjacency-list representation of G, describe an efficient algorithm to compute G^T. Write the pseudocode below.

Answer:

3.2 (0.1pt)

Adjacency List (Runtime): Analyze the running time of your algorithm for the adjacency-list representation.

Answer:

3.3 (0.4pt)

Adjacency Matrix (Pseudocode): Given the adjacency-matrix representation of G, describe an efficient algorithm to compute G^T. Write the pseudocode below.

Answer:

3.4 (0.1pt)

Adjacency Matrix (Runtime): Analyze the running time of your algorithm for the adjacency-matrix representation.

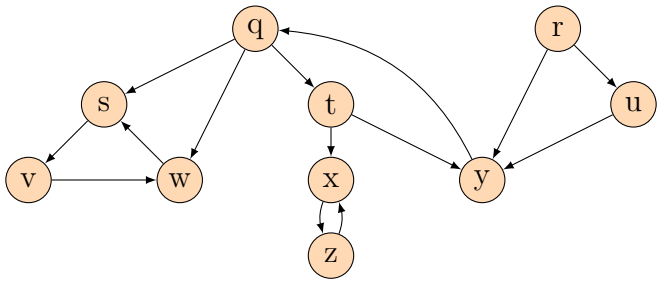
Answer:

4 Strongly Connected Components (1pt)

Consider the procedure `STRONGLY-CONNECTED-COMPONENTS(G)` given below:

1. call `DFS(G)` to compute finish times $u.f$ for each vertex u
2. compute G^T
3. call `DFS`(G^T), but in the main loop of `DFS`, consider the vertices in order of decreasing $u.f$ (as computed in line 1)
4. output the vertices of each tree in the depth-first forest formed in line 3 as a separate strongly connected component

Run this procedure on the graph shown below.



Assume that the main loop of `DFS` (lines 5-7) considers vertices in alphabetical order $(q, r, s, t, u, v, w, x, y, z)$ and that the adjacency lists are also sorted alphabetically.

4.1 (0.5pt)

List the finish time $u.f$ for each vertex u computed in line 1.

💡 **Format:** Write your answer as a list of `vertex: time` pairs, separated by commas. Sort the vertices alphabetically. **Example:** `a: 1, b: 2, c: 3`

Answer:

4.2 (0.5pt)

List the strongly connected components produced by the algorithm.

💡 **Format:** Write each component as a set of vertices enclosed in braces. List the components in the order they are discovered.
Example: `{a, b}, {c}, {d, e, f}`

Answer:

5 Minimum Spanning Tree (1pt)

💡 This problem is adapted from Exercise 21.1-3 in *Introduction to Algorithms* 4th edition.

Show that if an edge (u, v) is contained in some minimum spanning tree, then it is a light edge crossing some cut of the graph.

Answer:

6 Prim's Algorithm (1 pt)

💡 This problem is adapted from Exercise 21.2-2 in *Introduction to Algorithms* 4th edition.

🔧 Min-Priority Queue

A **min-priority** queue
(with a **binary min-heap** implementation)
is a data structure that
provides the following operations —

- BUILD-QUEUE(V) — build a queue in $O(|V|)$ time.
- EXTRACT-MIN(Q) — remove and return the element with the lowest `key` in $O(\log |V|)$ time.
- DECREASE-KEY$(Q, u, \texttt{newKey})$ — reduce the `key` of u to `newKey` in $O(\log |V|)$ time.

💡 The min-priority queue helps us to find the light edge to add.

Give a simple implementation of Prim's algorithm that runs in $O(V^2)$ time when the graph $G = (V, E)$ is represented as an adjacency matrix.

6.1 (0.6 pt)

Complete the following pseudocode by filling in the body of the `while` loop.

Prim-Matrix(G, w, r)
let $B[1..|V|]$ be a new bit array

```
for each u ∈ G.V do
    u.key = ∞
    u.π = NIL
    B[u] = True
end for
r.key = 0
Q = G.V
while Q ≠ ∅ do
                                        ▷ Fill in the missing part here
end while
```

Answer:

6.2 (0.4 pt)

Explain why the implementation above takes $O(V^2)$ time.

Answer:

7 Bellman-Ford Algorithm (1pt)

Bellman-Ford's Algorithm

```
Bellman-Ford(G, w, s)
INIT-SINGLE-SOURCE(G, s)
for i = 1 to |G.V| − 1 do
    for each edge (u, v) ∈ G.E do
        RELAX(u, v, w)
    end for
end for
for each edge (u, v) ∈ G.E do
    if v.d > u.d + w(u, v) then
        return False                    ▷ There is a negative cycle
    end if
end for
return True
```

This problem is adapted from Exercise 21.1-1 in *Introduction to Algorithms* 4th edition.

Consider the directed graph defined by the adjacency list and weights below. Run the Bellman-Ford algorithm on this graph using z as the source vertex.

Relax edges in the following order:

(z, a), (z, d), (a, c), (a, d), (b, z), (b, d), (c, a), (d, b)

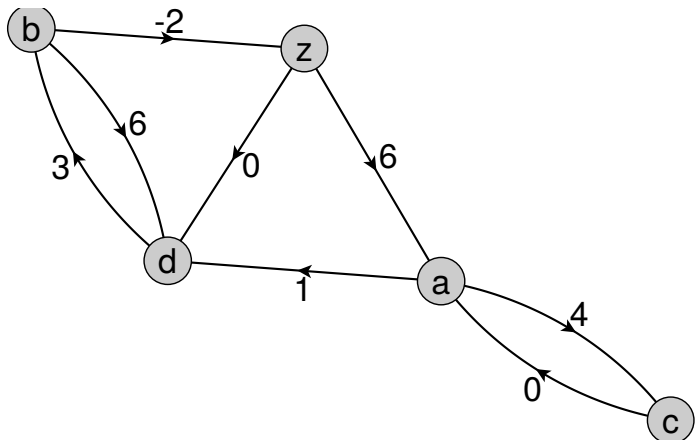


Figure 2: The Directed Graph

7.1 (0.4 pt)

Fill in the d values (shortest path estimates) after each round of relaxation. Initialization is Round 0.

Round	z	a	b	c	d
0	0	∞	∞	∞	∞
1					
2					
3					
4					

7.2 (0.4 pt)

Fill in the π values (predecessors) after each round of relaxation. Use NIL for no predecessor.

Round	z	a	b	c	d
0	NIL	NIL	NIL	NIL	NIL
1					
2					
3					
4					

7.3 (0.2 pt)

What value does the Bellman-Ford algorithm return?

Answer:

- ☐ True
- ☐ False

8 Dijkstra's Algorithm (1pt)

💡 This problem is adapted from Exercise 24.3-4 in *Introduction to Algorithms* 4th edition.

Consider the directed graph defined by the adjacency list and weights below. Run Dijkstra's algorithm on this graph using z as the source vertex.

Adjacency list (neighbor, weight): listed in alphabetical order by neighbor.

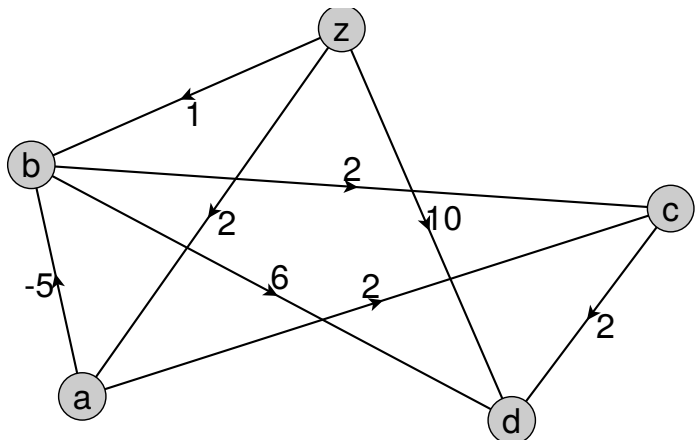


Figure 3: The Directed Graph

8.1 (0.4 pt)

Give the final d values (shortest path estimates) after Dijkstra's algorithm finishes.

	z	a	b	c	d
Final					

8.2 (0.4 pt)

Give the final π values (predecessors) after Dijkstra's algorithm finishes. Use NIL for no predecessor.

	z	a	b	c	d
Final					

8.3 (0.2 pt)

Does Dijkstra's algorithm produce the correct shortest path weights for this graph?

Answer:

- ☐ True
- ☐ False

⭐🌻🌷✨🌻🌷🐰🐰❤️⭐ **The End of The Quiz** ⭐🌻🌷✨🌻🌷🐰🐰❤️⭐